Calcolo Numerico

Marco Pittarello

Contents

1 Sistemi non decimali **2**
1.1 Conversione della parte intera . . . 2
1.2 Conversione della parte frazionaria . . . 3

2 Numeri macchina **4**
2.1 Interi macchina . . . 4
2.1.1 Overflow . . . 5
2.2 Floating point e standard IEEE 754 . . . 5
2.3 Tabella riassuntiva IEEE 754 . . . 6
2.4 Arrotondamento e troncamento . . . 6

3 Errore e norme **7**
3.1 Errore assoluto e relativo . . . 7
3.2 Norma euclidea . . . 7
3.3 Norma p . . . 7
3.4 Norma infinito . . . 8
3.5 Equivalenza delle norme . . . 8
3.6 Errore relativo . . . 8
3.7 Floating point: successivo e precedente . . . 8
3.8 Errore di rappresentazione . . . 9
3.8.1 Errore dovuto al troncamento . . . 9
3.8.2 Errore dovuto all'arrotondamento . . . 9

4 Precisione di macchina **9**
4.1 Operazioni macchina . . . 9
4.2 Errore relativo della moltiplicazione . . . 10
4.3 Errore relativo della somma . . . 10
4.4 Stabilità degli algoritmi . . . 11
4.5 Condizionamento dei problemi . . . 12

5 Algebra Lineare Numerica **13**

1 Sistemi non decimali

Definizione 1: Conversione da base N a base M

Dato $x > 0$ e nota la rappresentazione $(x)_N$, vogliamo trovare la rappresentazione $(x)_M$.
Scriviamo:

$$\begin{aligned} x &= x_n N^n + x_{n-1} N^{n-1} + \cdots + x_0 + x_{-1} N^{-1} + \cdots + x_{-r} N^{-r} \\ &= y_m M^m + y_{m-1} M^{m-1} + \cdots + y_0 + y_{-1} M^{-1} + \cdots + y_{-q} M^{-q}. \end{aligned}$$

Si procede separatamente per la parte intera e per la parte frazionaria.

1.1 Conversione della parte intera

Per convertire la parte intera si itera lo schema della divisione intera per M:

$$\begin{aligned} y_0 &:= x \bmod M & x &= q_0 M + y_0 \\ y_1 &:= q_0 \bmod M & q_0 &= q_1 M + y_1 \\ &\vdots \\ y_m &:= q_{m-1} \bmod M & q_{m-1} &= y_m < M \end{aligned}$$

OSSERVAZIONE: $x \bmod M$ indica il resto della divisione intera di x per M.

I resti ottenuti sono le cifre della rappresentazione in base M, lette dal basso verso l'alto. Pertanto:

$$(x_{\text{int}})_M = y_m y_{m-1} \dots y_1 y_0$$

Esempio 1: Conversione da base 2 a base 10

Convertiamo

$$(10011010010)_2$$

in base 10.
Si ha:

$$\begin{aligned} (10011010010)_2 &= 1 \cdot 2^{10} + 0 \cdot 2^9 + 0 \cdot 2^8 + 1 \cdot 2^7 + 1 \cdot 2^6 \\ &\quad + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 \\ &= 2^{10} + 2^7 + 2^6 + 2^4 + 2^1 \\ &= 1024 + 128 + 64 + 16 + 2 \\ &= 1234 \end{aligned}$$

Quindi:

$$\boxed{(10011010010)_2 = (1234)_{10}}$$

Esempio 2: Conversione da base 10 a base 2

Convertiamo

$$(1234)_{10}$$

in base 2 utilizzando divisioni successive per 2:

$$\begin{aligned}
1234 &= 617 \cdot 2 + 0 \\
617 &= 308 \cdot 2 + 1 \\
308 &= 154 \cdot 2 + 0 \\
154 &= 77 \cdot 2 + 0 \\
77 &= 38 \cdot 2 + 1 \\
38 &= 19 \cdot 2 + 0 \\
19 &= 9 \cdot 2 + 1 \\
9 &= 4 \cdot 2 + 1 \\
4 &= 2 \cdot 2 + 0 \\
2 &= 1 \cdot 2 + 0 \\
1 &= 0 \cdot 2 + 1
\end{aligned}$$

Leggendo i resti dal basso verso l'alto otteniamo:

$$10011010010$$

Pertanto:

$$\boxed{(1234)_{10} = (10011010010)_2}$$

1.2 Conversione della parte frazionaria

Per convertire la parte frazionaria si procede moltiplicando successivamente per M.

Sia

$$x_{\text{fraz}} \in [0, 1).$$

Si calcola:

$$Mx_{\text{fraz}} = y_{-1} + r_1, \qquad 0 \leq r_1 < 1,$$

dove y_{-1} è la parte intera del risultato.

Si ripete poi il procedimento:

$$Mr_1 = y_{-2} + r_2,$$

$$Mr_2 = y_{-3} + r_3,$$

e così via.

In generale:

$$Mr_k = y_{-(k+1)} + r_{k+1}, \qquad 0 \leq r_{k+1} < 1.$$

Se per qualche k si ottiene

$$r_k = 0,$$

il procedimento termina e la rappresentazione frazionaria è finita.

Le cifre ottenute, questa volta, si leggono dall'alto verso il basso:

$$(x_{\text{fraz}})_M = (0.y_{-1}y_{-2}y_{-3}\dots)_M.$$

Esempio 3: Conversione di una frazione decimale in binario

Convertiamo

$$(0.0625)_{10}$$

in base 2.
Moltiplichiamo successivamente per 2:

$$\begin{aligned} 0.0625 \cdot 2 &= 0.125 &&\longrightarrow 0 \\ 0.1250 \cdot 2 &= 0.250 &&\longrightarrow 0 \\ 0.2500 \cdot 2 &= 0.500 &&\longrightarrow 0 \\ 0.5000 \cdot 2 &= 1.000 &&\longrightarrow 1 \end{aligned}$$

La parte frazionaria successiva è 0, quindi il procedimento termina.
Pertanto:

$$\boxed{(0.0625)_{10} = (0.0001)_2}$$

Esempio 4: Un numero frazionario periodico

Consideriamo il numero

$$(0.1)_{10}.$$

Procedendo come sopra:

$$\begin{aligned} 0.1 \cdot 2 &= 0.2 &&\longrightarrow 0 \\ 0.2 \cdot 2 &= 0.4 &&\longrightarrow 0 \\ 0.4 \cdot 2 &= 0.8 &&\longrightarrow 0 \\ 0.8 \cdot 2 &= 1.6 &&\longrightarrow 1 \\ 0.6 \cdot 2 &= 1.2 &&\longrightarrow 1 \\ 0.2 \cdot 2 &= 0.4 &&\longrightarrow 0 \end{aligned}$$

e da questo punto la sequenza si ripete.
Quindi:

$$\boxed{(0.1)_{10} = (0.0001100110011\dots)_2}$$

Il numero ha quindi una rappresentazione binaria infinita e periodica.
OSSERVAZIONE: Un calcolatore binario non può rappresentare esattamente, in generale, un numero come 0.1 utilizzando un numero finito di bit.

2 Numeri macchina

2.1 Interi macchina

I tipi interi macchina più comuni utilizzano 32 o 64 bit, ad esempio `int32` e `int64`.

NOTA: Gli interi macchina a 32 o 64 bit non fanno parte dello standard IEEE 754, che riguarda invece la rappresentazione dei numeri in virgola mobile.

Se il primo bit è riservato al segno:

$$0 \longrightarrow + \qquad 1 \longrightarrow -$$

Con n bit e rappresentazione segno-modulo, il massimo intero positivo rappresentabile è:

$$I_{\max,n} = (011\ldots1)_2 = \sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1$$

Per una rappresentazione senza segno, invece:

$$I_{\max,n} = (111\ldots1)_2 = \sum_{i=0}^{n-1} 2^i = 2^n - 1$$

2.1.1 Overflow

Si verifica un overflow quando il risultato di un'operazione non è rappresentabile con il numero di bit disponibili.

Esempio 5: Overflow con interi macchina

Consideriamo interi macchina a 4 bit in rappresentazione segno-modulo.
Calcoliamo:

$$7 + 2$$

Abbiamo:

$$7 = (0111)_2 \qquad 2 = (0010)_2$$

La somma produce:

$$\begin{array}{r} 0111 \\ 0010 \\ \hline 1001 \end{array}$$

Il risultato ottenuto è:

$$(1001)_2 = -1$$

nella rappresentazione segno-modulo.
Quindi la macchina produce:

$$\boxed{7 + 2 = -1}$$

anziché 9.
Questo fenomeno prende il nome di overflow.

2.2 Floating point e standard IEEE 754

Usando la notazione binaria, un numero reale $x \neq 0$ può essere scritto, in forma normalizzata, come:

$$(x)_2 = (-1)^s \cdot 2^{e-b} \cdot (1.f)$$

dove:

- s è il bit di segno;
- e è l'esponente memorizzato;
- b è il bias, cioè la deviazione utilizzata per memorizzare anche esponenti negativi;
- $e - b$ è l'esponente effettivo;
- $1.f$ è la significanda (spesso chiamata informalmente mantissa).

Per un numero normalizzato, il bit iniziale 1 è sempre presente e quindi non è necessario memorizzarlo esplicitamente: si parla di hidden bit.

Il valore 0 richiede invece una convenzione particolare.

2.3 Tabella riassuntiva IEEE 754

	32 bit	**64 bit**
Bit totali	32	64
Bit segno	1	1
Bit esponente	8	11
Bit frazione	23	52
Bias b	127	1023
Intervallo e normalizzato	$0 < e < 255$	$0 < e < 2047$
$e = 0,\ f = 0$	± 0	± 0
$e = 0,\ f > 0$	denormalizzato	denormalizzato
$e = 255,\ f = 0$	$\pm\infty$	$\pm\infty$
$e = 255,\ f > 0$	NaN	NaN

2.4 Arrotondamento e troncamento

Quando il numero possiede più cifre di quante ne possano essere memorizzate, è necessario decidere cosa fare delle cifre in eccesso.

Definizione 2: Troncamento

Con il troncamento, le cifre oltre la precisione disponibile vengono semplicemente eliminate.

Definizione 3: Arrotondamento

Con l'arrotondamento, si considera la prima cifra che non può essere memorizzata e si modifica l'ultima cifra memorizzabile in funzione del suo valore.
Nel caso binario:
- se la cifra successiva è 1, si aumenta di 1 l'ultima cifra memorizzata;
- se la cifra successiva è 0, l'ultima cifra non viene modificata.

3 Errore e norme

Definizione 4: Approssimazione

Supponiamo di voler rappresentare o calcolare un valore $x \in \mathbb{R}$ (o $\in \mathbb{R}^m$), ma invece calcolo o ottengo un'approssimazione:

$$\widetilde{x} \approx x$$

3.1 Errore assoluto e relativo

Se $x \in \mathbb{R}$, definiamo l'errore assoluto come:

$$\boxed{|\widetilde{x} - x|}$$

Se, invece, $x \in \mathbb{R}^n$, per rappresentare l'errore assoluto, si utilizza:

$$\boxed{\|\widetilde{x} - x\|}$$

Dove $\|\cdot\|$ è una norma su $\mathbb{R}^m$.

Se $x \in \mathbb{R} \setminus \{0\}$, allora definisco l'errore relativo come:

$$\boxed{\frac{|\widetilde{x} - x|}{|x|}}$$

Definizione 5: Norma

Sia V uno spazio vettoriale e sia:

$$\|\cdot\| : V \longrightarrow \mathbb{R}_{\geq 0}$$

La funzione $\|\cdot\|$ è una norma se soddisfa, per ogni $u, v \in V$ e per ogni $\lambda \in \mathbb{R}$:

1. **omogeneità**: $\|\lambda v\| = |\lambda| \, \|v\|$
2. **definitezza**: $\|v\| = 0 \implies v = 0_V$
3. **disuguaglianza triangolare**: $\|u + v\| \leq \|u\| + \|v\|$

3.2 Norma euclidea

Per

$$v = (v_1, \ldots, v_n)^T \in \mathbb{R}^n$$

la norma euclidea è definita da:

$$\boxed{\|v\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \left(\sum_{d=1}^{n} v_d^2 \right)^{1/2}}$$

3.3 Norma p

Per $p \geq 1$ si definisce la norma p come:

$$\boxed{\|v\|_p = \left(\sum_{d=1}^{n} |v_d|^p \right)^{1/p}}$$

3.4 Norma infinito

Definizione 6: Norma infinito

La norma infinito è definita come:

$$\|v\|_\infty := \max_i |v_i|$$

3.5 Equivalenza delle norme

Sia $\{v_k\}$ una successione in $\mathbb{R}^n$ e siano $\|\cdot\|_a,\ \|\cdot\|_b$ due norme.

In $\mathbb{R}^n$ tutte le norme sono equivalenti. In particolare:

$$\begin{aligned} & v_k \longrightarrow v \quad \text{rispetto a } \|\cdot\|_a \quad \left(\text{ossia } \lim_{k\to+\infty} ||v_k - v||_a = 0\right) \\ & \Updownarrow \\ & v_k \longrightarrow v \quad \text{rispetto a } \|\cdot\|_b \quad \left(\text{ossia } \lim_{k\to+\infty} ||v_k - v||_b = 0\right) \end{aligned}$$

3.6 Errore relativo

Per $v \in \mathbb{R}^m$ e $\tilde{v}$ appross. di v definiamo l'errore relativo rispetto alla norma $\|\cdot\|$ su $\mathbb{R}^m$ come:

$$\boxed{\mathrm{Err}_{\mathrm{rel}}(\widetilde{v}, \|\cdot\|) = \frac{\|\widetilde{v} - v\|}{\|v\|}}$$

3.7 Floating point: successivo e precedente

Poiché $\mathbb{F}_{32}$ e $\mathbb{F}_{64}$ sono insiemi finiti e ordinati, possiamo definire:

Definizione 7: Successivo e precedente

Il successivo di x:

$$\mathrm{succ}(x) = \min\{y \in \mathbb{F} : y > x\} \quad \forall x \in \mathbb{F} \setminus \{\max \mathbb{F}\}$$

Il precedente di x:

$$\mathrm{prec}(x) = \max\{y \in \mathbb{F} : y < x\} \quad \forall x \in \mathbb{F} \setminus \{\min \mathbb{F}\}$$

Quanto vale $\mathrm{succ}(x) - x$?

Pongo

$$x = (-1)^s \cdot 2^{e-b} \cdot 1.f$$

$$\mathrm{succ}(x) = \begin{cases} (-1)^s \cdot 2^{e-b} \cdot 1.f + (-1)^s \cdot 2^{e-b} \cdot 0.\overbrace{00\ldots0}^{m-1}1 & , s = 0 \\ x - (-1)^s \cdot 2^{e-b} \cdot 0.\overbrace{00\ldots0}^{m-1}1 & , s = 1 \end{cases}$$

Dato che f contiene n bit, allora la distanza tra due numeri floating point consecutivi, nello stesso intervallo di esponente, è:

$$\boxed{\mathrm{succ}(x) - x = 2^{e-b-n}}$$

3.8 Errore di rappresentazione

$$|x - \mathrm{fl}^T(x)| \leq \begin{cases} \mathrm{succ}(\widetilde{x}) - \widetilde{x} & x > 0 \\ \widetilde{x} - \mathrm{succ}(\widetilde{x}) & x < 0 \end{cases} = |\mathrm{succ}(\widetilde{x}) - \widetilde{x}| = 2^{e-b-n}$$

3.8.1 Errore dovuto al troncamento

$$\frac{\left|x - \mathrm{fl}^T(x)\right|}{|x|} \leq \frac{\left|x - \mathrm{fl}^T(x)\right|}{\left|\mathrm{fl}^T(x)\right|} \leq \frac{2^{e-b-n}}{2^{e-b} \cdot 1.f}$$

$$\leq \frac{2^{e-b-n}}{2^{e-b} \cdot 1.0} = \boxed{2^{-n}}$$

Pertanto:

$$\boxed{\frac{\left|x - \mathrm{fl}^T(x)\right|}{|x|} \leq 2^{-n}}$$

3.8.2 Errore dovuto all'arrotondamento

L'arrotondamento fa meglio perchè:

$$\left|x - \mathrm{fl}^A(x)\right| \leq \frac{1}{2}|\mathrm{succ}(\widetilde{x}) - \widetilde{x}| = \frac{1}{2} \cdot 2^{e-b-n} = 2^{e-b-n-1}$$

Dividendo per $|x| \geq 2^{e-b}$ otteniamo:

$$\left|\frac{x - \mathrm{fl}^A(x)}{x}\right| \leq \frac{2^{e-b-n-1}}{2^{e-b}} = 2^{-n-1}$$

Pertanto:

$$\boxed{\frac{\left|x - \mathrm{fl}^A(x)\right|}{|x|} \leq 2^{-n-1}}$$

4 Precisione di macchina

Definizione 8: Precisione macchina

Definiamo la precisione di macchina come il più piccolo numero positivo ϵ tale che, se sommato a 1 in virgola mobile, da un risultato diverso da 1.

$$\epsilon_{\mathrm{mach}} := \min\{\epsilon > 0 : \mathrm{fl}(1 + \epsilon) > 1\}$$

4.1 Operazioni macchina

Siano definite le operazioni esatte:

$$\begin{aligned} \Delta : \mathbb{R} \times \mathbb{R} &\longrightarrow \mathbb{R}, && (+, \cdot) \\ \mathbb{R} \times (\mathbb{R} \setminus \{0\}) &\longrightarrow \mathbb{R}, && (:) \end{aligned}$$

Operazione macchina associata $\triangle$:

$$\begin{aligned} \triangle \quad & \mathbb{R} \times \mathbb{R} \longrightarrow \mathbb{F} \\ & (x, y) \longrightarrow \triangle(x, y) = \mathrm{fl}\left(\mathrm{fl}(x) \Delta \cdot \mathrm{fl}(y)\right) \end{aligned}$$

4.2 Errore relativo della moltiplicazione

$$\boxed{\mathrm{Err}_{\mathrm{rel}}(x \odot y) = \frac{|x \odot y - xy|}{|xy|} = \frac{|\mathrm{fl}\left(\mathrm{fl}(x)\,\mathrm{fl}(y)\right) - xy|}{|xy|}}$$

N.B. : noi supponiamo che

$$\frac{|x \cdot y - \mathrm{fl}(x \cdot y)|}{|x \cdot y|} \leq \epsilon_{\mathrm{mach}} \begin{cases} 2^{-n} \\ 2^{-n-1} \end{cases}$$

$z = x \cdot y$

$$\begin{aligned}
\mathrm{Err}_{\mathrm{rel}}(x \odot y) &= \frac{|\mathrm{fl}\left(\mathrm{fl}(x) \cdot \mathrm{fl}(y)\right) - \mathrm{fl}(x \cdot y) + \mathrm{fl}(x \cdot y) - x \cdot y|}{|x \cdot y|} \\
&\leq \frac{|\mathrm{fl}(\mathrm{fl}(x) \cdot \mathrm{fl}(y)) - \mathrm{fl}(x \cdot y)|}{|x \cdot y|} + \frac{\overbrace{|\mathrm{fl}(x \cdot y) - x \cdot y|}^{\epsilon_{\mathrm{mach}}}}{|x \cdot y|} \\
&= \frac{\overbrace{|\mathrm{fl}\left(\mathrm{fl}(x) \cdot \mathrm{fl}(y)\right) - \mathrm{fl}(x) \cdot \mathrm{fl}(y)|}^{1} + \overbrace{|\mathrm{fl}(x) \cdot \mathrm{fl}(y) - \mathrm{fl}(x \cdot y)|}^{2}}{|x \cdot y|} + \epsilon_{\mathrm{mach}} \\
&\leq^{1} \frac{|\mathrm{fl}\left(\mathrm{fl}(x) \cdot \mathrm{fl}(y)\right) - \mathrm{fl}(x) \cdot \mathrm{fl}(y)|}{|\mathrm{fl}(x) \cdot \mathrm{fl}(y)|} \cdot \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y)|}{|x \cdot y|} \\
&\leq \epsilon_{\mathrm{mach}} \cdot \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y)|}{|x \cdot y|} \\
&\leq^{2} \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y) - x \cdot y + x \cdot y - \mathrm{fl}(x \cdot y)|}{|x \cdot y|} \\
&\leq \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y) - x \cdot y|}{|x \cdot y|} + \frac{|\mathrm{fl}(x \cdot y) - x \cdot y|}{|x \cdot y|} \leq \epsilon_{\mathrm{mach}} \\
&\leq \epsilon_{\mathrm{mach}} + \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y) - x \cdot y|}{|x \cdot y|} \\
1 + 2 &\leq \epsilon_{\mathrm{mach}} \left(1 + \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y)|}{|x \cdot y|}\right) + \frac{|\mathrm{fl}(x) \cdot \mathrm{fl}(y) - x \cdot y|}{|x \cdot y|} \\
&\leq \epsilon_{\mathrm{mach}} \left(1 + \epsilon_{\mathrm{mach}} \underbrace{(\ldots\ldots)}_{\text{dipende da } x,\, y}\right)
\end{aligned}$$

Al primo ordine, il comportamento dell'errore relativo non dipende dai valori specifici di x e y.
L'errore relativo della moltiplicazione macchina u: $\epsilon + M\epsilon$.
La moltiplicazione è quindi considerata stabile, ossia l'errore relativo in output è controllato da quello in input.

4.3 Errore relativo della somma

L'errore relativo della somma macchina è:

$$\boxed{\mathrm{Err}_{\mathrm{rel}}(x \oplus y) = \frac{|\mathrm{fl}\left(\mathrm{fl}(x) + \mathrm{fl}(y)\right) - (x + y)|}{|x + y|} = \frac{|x \oplus y - (x + y)|}{|x + y|}}$$

Se x e y hanno segno opposto e sono molto vicini tra loro, può verificarsi una forte cancellazione numerica, ossia la perdita di cifre significative.

Esempio 6: Cancellazione Numerica

Ipotesi in cui x e y si annullano fra loro lasciando solo un ϵ_{mach}:

$$y = -x + \epsilon_{\mathrm{mach}}$$

Svolgimento:

$$\begin{aligned}\mathrm{Err}_{\mathrm{rel}}(x \oplus y) &\leq \epsilon_{\mathrm{mach}} \left(1 + \frac{|x|+|y|}{|x+y|} + \epsilon_{\mathrm{mach}} \frac{|x|+|y|}{|x+y|}\right) \\ &= \epsilon_{\mathrm{mach}} \left(1 + \frac{2|x| + \epsilon_{\mathrm{mach}}}{\epsilon_{\mathrm{mach}}} + 2|x| + \epsilon_{\mathrm{mach}}\right) \\ &= \left(\epsilon_{\mathrm{mach}} + 2|x| + \epsilon_{\mathrm{mach}} + 2|x|\epsilon_{\mathrm{mach}} + \epsilon_{\mathrm{mach}}^2\right) \\ &= 2\left(|x| + \epsilon_{\mathrm{mach}}\right) + \dots\end{aligned}$$

I $\dots$ assorbono $\epsilon_{\mathrm{mach}}^2$ e $2|x|\epsilon_{\mathrm{mach}}$ che sono numeri talmente piccoli rispetto agli altri da essere irrilevanti. Significa che se x è grande, anche l'errore relativo è enorme.

4.4 Stabilità degli algoritmi

Esempio 7: Calcolo dell'integrale col metodo iterativo

$$I_n = \frac{1}{e} \int_0^1 e^x \cdot x^n \cdot dx$$

Svolgimento: Conoscendo il valore di I_0 ossia:

$$I_0 = \frac{1}{e} \int_0^1 e^x \cdot dx = \frac{1}{e} \left[e^x\right]_0^1 = \frac{e-1}{e}$$

Inoltre si può integrare per parti I_n:

$$\begin{aligned}I_n &= \frac{1}{e} \left[\left(e^x x^n\right)_0^1 - \int_0^1 e^x \cdot n x^{n-1} \cdot dx\right] \\ &= \frac{1}{e} \left[e - n \int_0^1 e^x \cdot x^{n-1} \cdot dx\right] \\ &= 1 - n \frac{1}{e} \int_0^1 e^x \cdot x^{n-1} \cdot dx \\ &= 1 - n \cdot I_{n-1}\end{aligned}$$

Di conseguenza conosco la successione definita per ricorsione:

$$\boxed{\begin{cases} I_0 = \frac{e-1}{e} \\ I_n = 1 - n \cdot I_{n-1} \end{cases}}$$

In realtà I_{n-1} viene calcolato con un errore, quindi supponiamo di aver calcolato $\widetilde{I}_{n-1} = I_{n-1} + \epsilon_{n-1}$. Quindi per calcolare I_n:

$$\begin{aligned}I_n &= 1 - n \cdot I_{n-1} \\ \widetilde{I}_n &= 1 - n \cdot \widetilde{I}_{n-1} = \underbrace{1 - n I_{n-1}}\, - n\epsilon_{n-1} \\ &= \boxed{I_n - n\epsilon_{n-1}}\end{aligned}$$

Questo risultato è iterabile dato che si verifica per ogni n, inoltre so che $\epsilon_n = -n\epsilon_{n-1}$.
Iterando n volte ottengo:

$$\boxed{\epsilon_n = (-1)^n \cdot n! \cdot \epsilon_0}$$

L'errore assoluto è quindi:

$$|\widetilde{I}_n - I_n| = n! \cdot |\epsilon_0| = n! \cdot \mathrm{fl}\left(\frac{e-1}{e}\right) - \frac{e-1}{e}$$

Da cui ottengo:

$$= n! \cdot \underbrace{\frac{|\,\mathrm{fl}\left(\frac{e-1}{e}\right) - \frac{e-1}{e}|}{\frac{e-1}{e}}}_{\leq \epsilon_{\text{mach}}} \cdot \underbrace{\frac{e-1}{e}}_{\leq 1}$$

La frazione è l'errore relativo di rappresentazione di un numero reale
L'errore assoluto è dominato da:

$$\boxed{\leq n! \cdot \epsilon_{\text{mach}}}$$

Adesso stimiamo quanto è I_n:

$$I_n = \frac{1}{e}\int_0^1 x^n e^x \cdot dx = O(\frac{1}{n})$$

Dimostriamo che è vero:

Posso stimarlo da sopra con un integrale $\leq$

$$nI_n = \frac{1}{e}\int_0^1 nx^n e^x \cdot dx \leq \frac{1}{e}\int_0^1 enx^n \cdot dx = \frac{n}{n+1}\left[x^{n+1}\right]_0^1 = \frac{n}{n+1}$$

E posso stimarlo da sotto con un'integrale $\geq$

$$nI_n = \frac{1}{e}\int_0^1 nx^n e^x \cdot dx \geq \frac{1}{e}\int_0^1 nx^n \cdot dx = \frac{n}{n+1}\cdot\frac{1}{e}$$

Abbiamo quindi scoperto che $I_n \approx \frac{1}{n+1}$
Adesso possiamo dire che l'errore relativo:

$$\frac{|\widetilde{I}_n - I_n|}{|I_n|} \leq \frac{n!\epsilon_{\text{mach}}}{\frac{1}{n+1}} = (n+1)n!\epsilon_{\text{mach}} = \boxed{(n+1)!\epsilon_{\text{mach}}}$$

Questo algoritmo si dice **instabile**.

Definizione 9: Algoritmo Stabile

Un algoritmo si dice **stabile** se non amplifica in modo incontrollato gli errori presenti sui parametri di input e i dati. In generale, si dice **stabile** un algoritmo per cui vale:

$$\mathrm{Err}^{\mathrm{rel}}(\text{output}) \leq C_{\text{stab}}\,\mathrm{Err}^{\mathrm{rel}}(\text{input})$$

Dove C_{stab} è la costante di stabilità.

4.5 Condizionamento dei problemi

Definizione 10: Problema ben posto

Un problema si dice **ben posto** se:

1. $\forall d \in D \quad \exists\, x^x \in X :\ \mathrm{F}(x^x, d) = 0$ esiste una **soluzione**
2. x^x, la soluzione è **unica**
3. x^x varia con **continuità** rispetto ai dati

La soluzione è data da:

$$d \mapsto x^x(d)\ :\ \mathrm{F}(x^x(d), d) = 0 \quad x^x\ :\ D \to X$$

Condizionamento di un problema ben posto:

$$\frac{\text{cond. locale}}{\text{assoluto}} \quad \mathrm{k}(d) := \limsup_{\widetilde{d}\to d} \frac{\left|x(d) - x(\widetilde{d})\right|}{\left|d - \widetilde{d}\right|}$$

$$\limsup_{\widetilde{d}\to d} f(\widetilde{d}) = \limsup_{\to 0^+}$$

5 Algebra Lineare Numerica